

UNSUPERVISED LEARNING

let's represent the MNIST digits dataset test:

$f: X \rightarrow Y$
 $\uparrow \quad \uparrow$
pictures of digits $\{0, 1, \dots, 9\}$

$S = (\text{img}_1, \cancel{f(\text{img}_1)}), (\text{img}_2, \cancel{f(\text{img}_2)}), \dots, (\text{img}_{1000}, \cancel{f(\text{img}_{1000})})$
 $\uparrow$
TRAINING SET

In UNSUPERVISED LEARNING, we will NOT HAVE SAMPLES *
ANYMORE. We will analyze the problem of CLUSTERING,
where instances are all grouped for similarity.

CLUSTERING

- GROUP SIMILAR OBJECTS (points)
 - SEPARATE DISSIMILAR OBJECTS
- CAN BE CONTRADICTORY *

There is NO UNIQUE FORMULATION for CLUSTERING (we will see an example that tries, failing)

* Consider an example with many similar points:
 $x_1, x_2, \dots, x_m$; x_i is similar to $x_{i+1} \forall i$

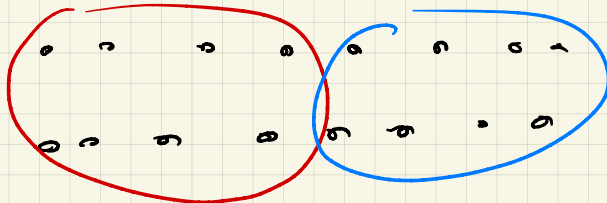
So, subsequent points are similar, but distant are not. But we said that similarities must be grouped. So we can't group x_1, x_m even though they respect similarity.

In clustering, there are a few possible approaches.

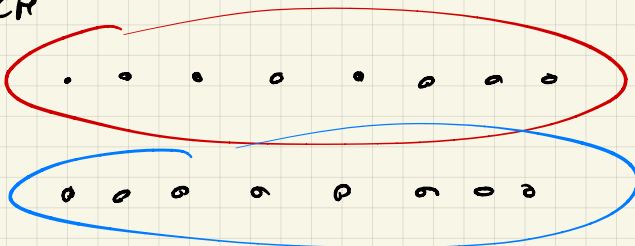
EXAMPLE

Given a set of points, I want to not separate closed-by points:

1ST APPROACH



2ND APPROACH



Both are correct!

THERE IS NOT A SINGLE CORRECT
APPROACH

FORMAL MODEL FOR CLUSTERING

INPUT:

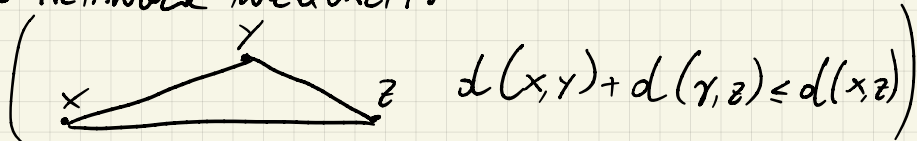
- Instance space: $X \subseteq X'$

- Distance function: $d: X \times X \rightarrow \mathbb{R}_+$

• $d(x, x) = 0 \quad \forall x \in X$

• $d(x, y) = d(y, x), \quad \forall x, y \in X$

• TRIANGLE INEQUALITY



OUTPUT:

- $C = (C_1, \dots, C_k)$

• PARTITION SATISFACTION:

FOR SOME ALGORITHMS, THERE
ARE MORE POSSIBLE COR-
RECT CLUSTERINGS

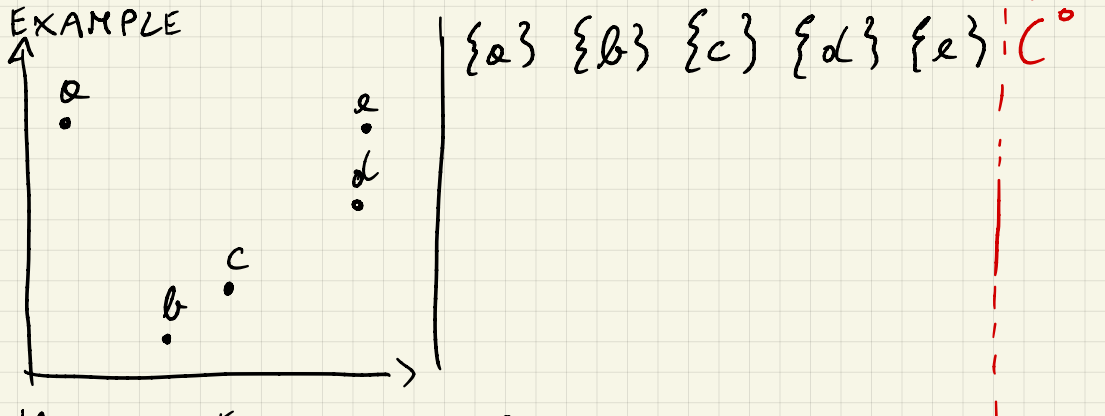
(OR DENDROGRAM)

• $\bigcup_{i=1}^k C_i = X$
 TREE-BASED REPRESENTATION
 OF CLUSTERS
 $C_i \cap C_j = \emptyset$ for $i \neq j$

LINKAGE-BASED CLUSTERING (outputs DENDROGRAM)

- Start with clustering $C_i = \{x_i\}$ for $x_i \in X$
↳ EVERY POINT IS A CLUSTER
- Repeat until TERMINATION CONDITION:
 - Merge "closest" clusters of current partition
- Return current partition

EXAMPLE

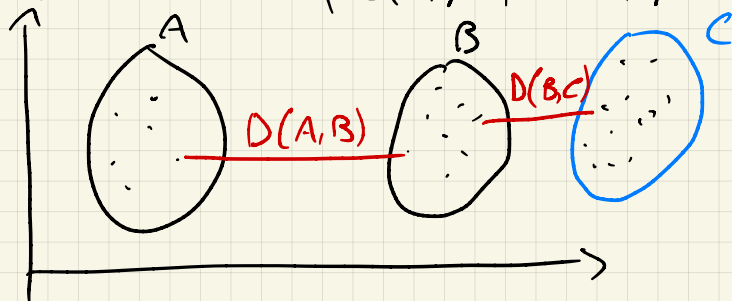


How do I proceed now?

Let's define BETWEEN-CLUSTER DISTANCE $D(A, B)$

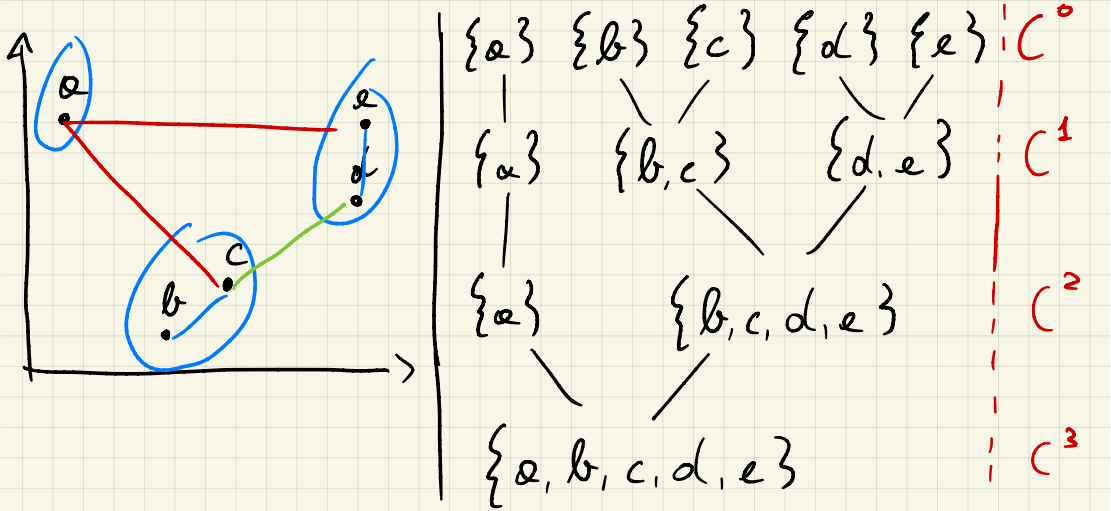
SINGLE LINKAGE CLUSTERING

$$D(A, B) = \min (d(x, y) \mid x \in A, y \in B)$$



- Then merge for A and B (not the example ones, in the example we should merge B and C)

BACK TO THE EXAMPLE

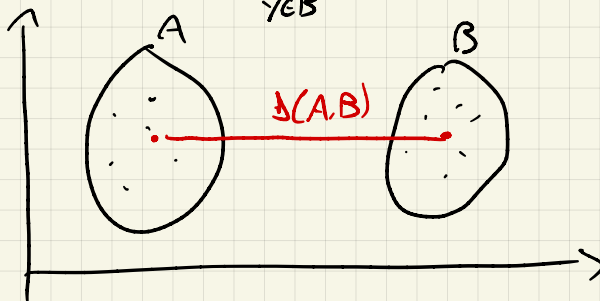


If you don't consider any TERMINATION condition, you will always have C^3 as final partition. However, your output will be a DENDROGRAM, so you could pick any C^i you want.

This way, the algorithm is VERY EXPENSIVE (w/out any TERMINATION COND.)

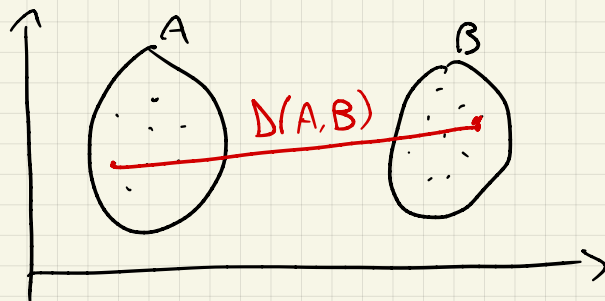
• AVERAGE LINKAGE CLUSTERING

$$D(A, B) = \frac{1}{|A||B|} \sum_{\substack{x \in A \\ y \in B}} d(x, y)$$



• MAX LINKAGE CLUSTERING

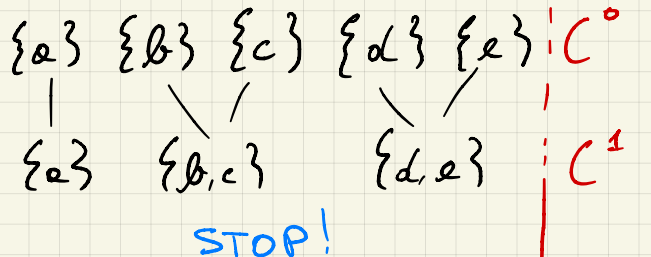
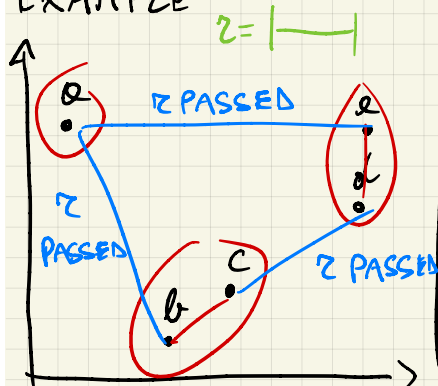
$$D(A, B) = \max \{d(x, y) | x \in A, y \in B\}$$



TERMINATION CONDITION

- # of clusters (provided as input)
- DISTANCE UPPER BOUND:
 - All between-cluster distance pass DISTANCE THRESHOLD
 - ... other bounds can be implemented

EXAMPLE



However, linkage-based clustering is EXPENSIVE in complexity
 $\sim O(n^2)$

Let's see other possible types of clustering.

K-MEANS CLUSTERING

- Clustering is seen as an OPTIMIZATION PROBLEM

INPUT: (X, d) $X \subseteq X'$ (distance function d on X')

- # of clusters K

OUTPUT:

- Partition $C = (C_1, \dots, C_K)$ (into K clusters)

G : objective function (evaluates how good the partition is)

- INPUT:

- dataset (with d) (X, d)
- $C_1, \dots, C_K$

- OUTPUT:

- value from $\mathbb{R}_+$

K-MEANS (and variants) searches for $C = (C_1, \dots, C_K)$ that minimizes $G((X, d), (C_1, \dots, C_K))$

K-MEANS OPTIMIZATION PROBLEM: find $C_1, \dots, C_K$ s.t.
 $G((X, d), (C_1, \dots, C_K))$ is minimal

$$C = \underset{(C_1, \dots, C_K)}{\operatorname{argmin}} G((X, d), (C_1, \dots, C_K))$$

To focus on understanding the DBS. FUNCTION, we need to introduce CENTROID

CENTROID

For a cluster C_i , it is defined a CENTROID a function:

$$\mu_i = \underset{\mu \in X'}{\operatorname{argmin}} \sum_{x \in C_i} d(x, \mu)^2$$

(if d is euclidean distance: $\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$)

It is the function that minimizes the squared distance between all points in a cluster

Now, we can define G

K-MEANS OBJECTIVE FUNCTION G EUCLIDEAN DISTANCE

$$G_{\text{k-means}}((X, d), (C_1, \dots, C_k)) = \sum_{i=1}^k \sum_{x \in C_i} d(x, \mu_i(C_i))^2$$

This solution is very complex to find, so a lot of approximations will be used.

Let's see the algorithm model.

K-MEANS

INPUT: K : # of clusters; $X \subseteq \mathbb{R}^m$: feature space; d : EUCLIDEAN DIST.

- INITIALIZE: Randomly choose centroids $\mu_1, \dots, \mu_K$
- REPEAT UNTIL CONVERGENCE:

$$\forall i \in \{1, \dots, k\} \text{ let } C_i = \{x \in X \mid i = \underset{j}{\operatorname{argmin}} \|x - \mu_j\|\}$$

$$\forall i \in \{1, \dots, k\} \quad \mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$$

THESE ARE THE CENTROIDS AFTER ASSIGNING EVERY x TO A CLUSTER

So, you have the random centroid, you assign $\forall x \in X$ basing on centroids, then you compute the new centroids (based on the actual clusters), then you re-assign x to new clusters based on new centroids and so on and so forth until no changes can be made $\rightarrow$ you reach convergence

VARIANTS

• K-MEDOID

$$\text{MEDOID: } \mu_i(C_i) = \underset{\mu \in X}{\operatorname{argmin}} \sum_{x \in C_i} d(x, \mu)^2$$

• K-MEDIAN

$$\text{MEDIAN: } \mu_i(C_i) = \underset{\mu \in X}{\operatorname{argmin}} \sum_{x \in C_i} d(x, \mu)$$

N.B.: When you compute μ , you will pass through EVERY point in your dataset

LEAKA

Each iteration of K-MEANS will not increase the objective function.

So, our algorithm is truly dependent from the initial centroids (different initial centroids lead to different clusters after convergence).

NON-GUARANTEES

- We don't know the necessary # of iterations to converge
- NO APPROXIMATION GUARANTEE: I can't be sure to find a solution that is lower than an arbitrary bound (no local minimum guarantee)

What can we do to improve this algorithm?

(PARTIAL) SOLUTION: REPEATED EXECUTIONS

FURTHER ISSUE: Hyperparameter k (which is the right k ?)

- Test with different values of k

Returning to clustering, we still don't have a formalized structure for clustering. The attempts were numerous:

- KEELINBERG (2003): AXIOMATIZATION OF CLUSTERING FUNCTION

$$F((X, d)) = (C_1, \dots, C_k)$$

|| (DESIRABLE) PROPERTIES OF F :

- SCALE INVARIANCE: F returns the same partition after scaling the data
- RICHNESS: $\forall C_i \in (C_1, \dots, C_k) \exists d \text{ s.t. } F((X, d)) = (C_1, \dots, C_k)$
(so, distance function DETERMINES the output)
- CONSISTENCY: similar points will be clustered together and dissimilar points will be separated

We can prove an impossibility result:

IMPOSSIBILITY RESULT

$\nexists$ clustering function F that satisfies at the same time SCALE INVARIANCE, RICHNESS AND CONSISTENCY.

NEURAL NETWORKS

Initially, we approached to the optimization problem parametrically by minimizing the expected risk (or loss function).

$$f: X \rightarrow Y \quad S' = \{(x_1, t_1), \dots, (x_m, t_m)\} \quad t_i = f(x_i)$$

$$H \quad h(x) = h(x; \vartheta) = w_1 x_1 + w_2 x_2 + w_0 \quad \vartheta = \{w_1, w_2, w_0\}$$

$L(h_S)$ is the loss function or ERM